

Manual Q&A operator_guide GPT-5.4 Nano 구조 발표 정리

1. 목표

이번 단계의 목표는 Manual Q&A가 더 이상 Python 코드에서 정해진 답변을 조립하지 않고, 실제 LLM이 CSV 근거를 참고해서 플레이어에게 답변하도록 만드는 것입니다.

프로토타입의 최종 흐름은 아래와 같습니다.

```
플레이어 질문
-> AgentPipeline
-> Orchestrator LLM
-> operator_guide 선택
-> operator_guide leaf agent 선택
-> CSV 기반 질문 분류 및 근거 수집
-> system prompt + user prompt 생성
-> gpt-5.4-nano 호출
-> JSON 답변 반환
-> Unreal UI 표시
```

2. 전체 구조

플레이어가 질문을 입력하면 backend의 `AgentPipeline`이 요청을 받습니다.

이후 Orchestrator가 어떤 agent가 처리해야 하는지 판단합니다.

Manual Q&A 질문이면 Orchestrator는 `operator\_guide`를 선택합니다.

```
AgentPipeline
-> route_top_agent
-> selectedAgent = operator_guide
```

그 다음 `operator\_guide` 내부에서 질문 성격에 맞는 leaf agent를 선택합니다.

```
operator_guide.recipe_explainer
operator_guide.machine_help
operator_guide.troubleshooter
```

예를 들어 "컨베이어가 멈췄는데 뭘 확인해야 해?" 같은 질문은 문제 해결 질문이므로 `operator\_guide.troubleshooter`가 선택됩니다.

3. CSV 기반 Manual Q&A 처리

leaf agent가 선택되면 `ManualQAService`가 질문을 처리합니다.

```
ManualQAService
-> ManualQAQuestionClassifier
-> ManualQAContextBuilder
```

```
-> CsvManualQARepository
-> ManualQAPromptBuilder
```

각 역할은 다음과 같습니다.

```
ManualQAQuestionClassifier
-> 질문을 equipment, resource, recipe, troubleshooting, unknown 유형으로 분류

CsvManualQARepository
-> equipment.csv, resources.csv, recipes.csv, troubleshooting_rules.csv, action_policy.csv 조회

ManualQAContextBuilder
-> 질문과 관련된 CSV row를 evidence JSON으로 정리

ManualQAPromptBuilder
-> system message와 user message를 생성
```

중요한 점은 CSV가 최종 답변 자체가 아니라, GPT가 답변을 만들기 위한 근거로 쓰인다는 것입니다.

4. LLM 설정

현재 시연용 production 설정은 `backend/.env.prod`를 사용합니다.

```
ENVIRONMENT=production
FACTORY_LLM_DEFAULT_PROVIDER=openai
FACTORY_LLM_DEFAULT_MODEL=gpt-5.4-nano
OPENAI_API_KEY=...
```

로컬 개발용 `env`는 Ollama local LLM을 사용합니다.

```
ENVIRONMENT=development
FACTORY_LLM_DEFAULT_PROVIDER=local
FACTORY_LLM_DEFAULT_MODEL=gemma4:e2b
FACTORY_LLM_DEFAULT_BASE_URL=http://localhost:11434/v1
```

즉, 개발 중에는 `gemma4:e2b`로 확인하고, 프로토 시연에서는 `gpt-5.4-nano`로 답변을 생성할 수 있습니다.

5. GPT에 들어가는 메시지 구조

최종 답변 생성 단계에서는 단순한 문자열 prompt가 아니라 chat message 구조를 사용합니다.

```
[
  {
    "role": "system",
    "content": "튜토리얼 오퍼레이터 정체성, 말투, CSV 근거 우선 규칙, JSON 출력 규칙"
  },
  {
    "role": "user",
    "content": "플레이어 질문, leaf agent, 질문 유형, CSV evidence, 출력 계약"
  }
]
```

system prompt는 `backend/src/agents/operator\_guide/system\_prompt.py`에 있습니다.

system prompt의 핵심 규칙은 다음과 같습니다.

1. Factory Space 안의 친절한 튜토리얼 오퍼레이터처럼 답변
2. 한국어로 답변
3. CSV evidence를 가장 중요한 근거로 사용
4. 근거에 없는 기계, 자원, 레시피, 수치, 규칙은 지어내지 않음
5. 근거가 부족하면 모른다고 안내
6. final_answer, actions, question, topic만 포함한 JSON 객체로 반환
7. actions는 항상 빈 배열

6. 입력 JSON 예시

Unreal 또는 Front에서 backend로 보내는 입력은 아래와 같습니다.

```
{
  "type": "agent.request",
  "request_id": "manual-qa-gpt5-demo-001",
  "session_id": "demo-session",
  "client_id": "unreal-client",
  "payload": {
    "question": "컨베이어가 멈췄는데 뭘 확인해야 해?"
  },
  "context": {
    "language": "ko",
    "mode": "prototype_demo"
  }
}
```

7. 실제 GPT-5.4 Nano 응답 예시

실제 `env.prod`의 `gpt-5.4-nano` 설정으로 호출했을 때 받은 응답은 아래와 같습니다.

```
{
  "type": "agent.response",
  "request_id": "manual-qa-gpt5-demo-001",
  "session_id": "demo-session",
  "client_id": "codex-test",
  "agent": "operator_guide",
  "payload": {
    "final_answer": "컨베이어가 멈추면 먼저 전력 상태부터 확인해 주세요(발전기와 전력 연결). 다음으로 입력 자원이 끊기지 않았는지 확인하고, 출력 저장 공간이 가득 차서 막히지 않았는지도 봐주세요. 그 다음 컨베이어 방향과 연결 상태가 올바른지 점검하면 좋아요. 마지막으로 장비에 설정된 레시피가 맞는지도 확인해 주세요. 이렇게 전력 → 입력 → 저장공간 → 컨베이어 → 레시피 순서로 보면 원인을 빠르게 좁힐 수 있어요.",
    "actions": [],
    "question": "컨베이어가 멈췄는데 뭘 확인해야 해?",
    "topic": "troubleshooting",
    "metadata": {
      "llm": "used",
      "llmSlot": "default",
      "llmProvider": "openai",
      "llmModel": "gpt-5.4-nano",
      "selectedAgent": "operator_guide",
      "selectedLeafAgent": "operator_guide.troubleshooter"
    }
  }
},
```

```
"streams": []
}
```

Unreal UI에서는 기본적으로 아래 값만 표시하면 됩니다.

```
payload.final_answer
```

8. 기존 룰베이스 방식과의 차이

이전 방식은 Python 코드가 답변 문장을 조립했습니다.

```
CSV 조회
-> response_builder.py
-> 정해진 template 문장 조립
-> final_answer 반환
```

현재 방식은 GPT가 CSV 근거와 system prompt를 받아 직접 답변을 생성합니다.

```
CSV 조회
-> CSV evidence JSON 생성
-> system/user messages 생성
-> gpt-5.4-nano 호출
-> LLM이 final_answer 작성
```

따라서 답변은 고정 template이 아니라, CSV 근거를 바탕으로 자연스럽게 생성됩니다.

9. GPT-5.4 Nano 대응 수정

`gpt-5.4-nano`는 OpenAI 호출에서 `max\_tokens` 대신 `max\_completion\_tokens`를 요구했습니다. 그래서 OpenAI adapter는 모델명이 `gpt-5`로 시작하면 아래 파라미터를 사용하도록 수정했습니다.

```
gpt-5* 모델
-> max_completion_tokens

그 외 모델
-> max_tokens
```

이 수정 후 실제 `gpt-5.4-nano` 호출이 성공했습니다.

10. 발표용 핵심 문장

이번 Manual Q&A 구조는 플레이어 질문을 Orchestrator가 operator_guide로 라우팅하고, operator_guide 내부 leaf agent가 질문 유형을 나눈 뒤, CSV 매뉴얼 데이터를 근거로 gpt-5.4-nano가 최종 답변을 생성하는 구조입니다.

즉, 답을 미리 정해놓은 룰베이스가 아니라, 게임 매뉴얼 CSV와 system prompt를 기반으로 실제 LLM이 플레이어에게 자연어 안내를 제공하는 방식입니다.